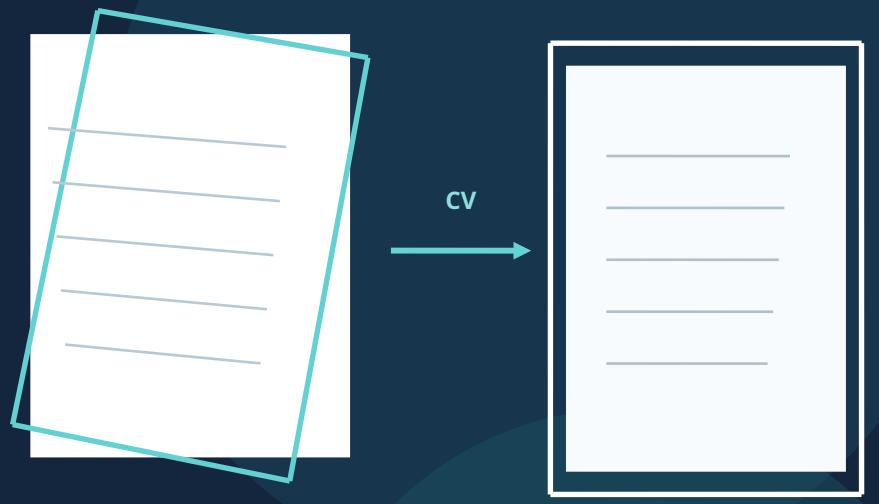
IMAGE PROCESSING & COMPUTER VISION

Smart Document Scanner

Document Detection & Perspective Correction Using Classical Image Processing

A backend-first, explainable pipeline that turns angled phone photos into clean, straightened scans.



DETECT → CORRECT → ENHANCE → EXPORT

Classical computer vision · Human-guided fallback · No model training

PROJECT TEAM

Saurabh Kumar Singh · **20233256**

Shubhranshu Mishra · **20233272**

Shivanshu Pathak · **20233267**

Suresh Choudhary · **20233281**

Submitted to **Dr. Abhimanyu Sahu**

A photo is not yet a scan

The system automates the cleanup steps that are usually done manually in a scanner app.

INPUT

Handheld photograph



Skew • Shadow • Uneven edges

OUTPUT

Straightened document



PERSPECTIVE FIXED

- Find the page boundary automatically
- Let the user adjust corners when needed
- Export as image or PDF

DESIGN CHOICE **Classical image processing keeps the pipeline explainable, lightweight, and easy to inspect step by step.**

One client, one API, a modular vision backend

The browser handles interaction; image understanding and transformation happen on the server.

CLIENT

React 18 + Vite

Upload · preview · corner editor

→

TRANSPORT

REST API

Multipart file + JSON

→

ENTRY POINT

FastAPI Router

Validate request · call service

→

BACKEND ENGINE

Vision Processing Pipeline

Validation → Detection → Correction → Enhancement → PDF

BACKEND SERVICE FLOW

01 **Validate** — Decode + EXIF

→

02 **Preprocess** — Resize + smooth

→

03 **Detect** — Masks + contours

→

04 **Perspective** — Order + warp

→

05 **Enhance** — B&W / gray / CLAHE

→

06 **Export** — PDF synthesis

←

Structured JSON for geometry and metrics · Image data or binary PDF for generated outputs

Key separation: UI state stays in React; all authoritative image processing stays in the FastAPI/OpenCV backend.

Clear API contracts, single-responsibility services

Each endpoint exposes one processing capability, while the unified route can run the complete pipeline.

REST ENDPOINTS

GET	/api/health	Server + OpenCV status
POST	/api/detect	Return corners, method and metrics
POST	/api/correct	Perspective-correct using 4 corners
POST	/api/enhance	Apply selected readability filter
POST	/api/process	Detect → correct → enhance
POST	/api/export/pdf	Return generated PDF stream

VISION SERVICE MODULES

image_validation.py
Decode, MIME checks, EXIF

preprocessing.py
Resize, blur, auto-Canny

document_detection.py
Segmentation + candidate scoring

perspective.py
Corner order + homography warp

enhancement.py
Adaptive threshold + CLAHE

pdf_export.py / metrics.py
PDF creation + quality measures

Schemas keep requests and responses explicit; errors are returned as useful HTTP status codes.

Prepare the image before searching for edges

A few careful input and preprocessing steps make later detection faster and more reliable.

INPUT GUARDRAILS

- **Format check**
  Confirm the uploaded bytes decode as an image.
- **Orientation**
  Apply EXIF rotation so visual and pixel coordinates agree.
- **Size limit**
  Reject uploads above the configured 15 MB limit.
- **Safe fallback**
  Return a clear 400/413 response instead of continuing with bad input.

PREPROCESSING SEQUENCE

1. **Downsample for detection** — Keep aspect ratio; cap the longest side at about 900 px.
2. **Build useful image views** — Use grayscale for luminance and HSV for paper-like color cues.
3. **Reduce sensor noise** — Gaussian smoothing removes tiny variations while preserving major edges.
4. **Suppress text inside the page** — Morphological closing fills narrow dark strokes without relying on heavy blur.

Important: downsampling is for detection only; perspective correction uses the original-resolution image.

Use multiple signals to find the page boundary

No single threshold works for every background, so the backend combines several segmentation methods.

HSV paper mask
Looks for bright, low-saturation regions

Otsu threshold
Tests normal and inverted masks

Adaptive threshold
Handles local shadows and uneven light

Color Canny
Finds edges that appear in color channels

Luminance Canny
Finds strong brightness boundaries

CLAHE + Canny
Enhances local contrast before edge search

Contour extraction + candidate generation

Trace contours
Follow connected boundary pixels

→

Simplify shape
Reduce hundreds of points

→

Extract 4 corners
Use rotated extrema if corners are rounded

→

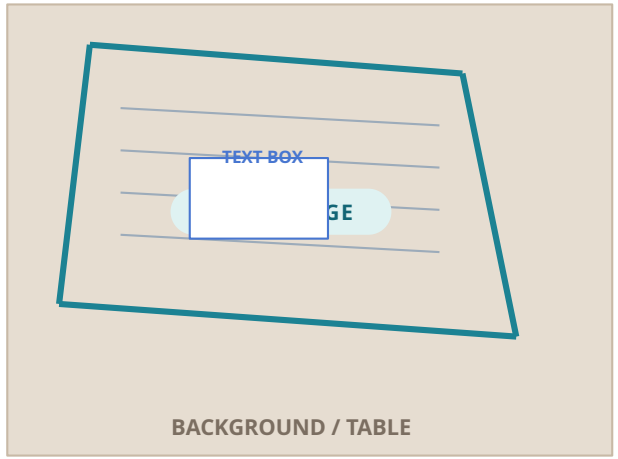
Refine coordinates
Sub-pixel adjustment near real edges

Output: candidate quadrilaterals, ready for scoring — not yet a final guess.

Choose the document, not just the largest shape

Contours can belong to a desk, a paragraph, or the camera frame. The scoring stage ranks plausible page candidates.

EXAMPLE CANDIDATES



COMBINED SCORING SIGNALS

- **Paper brightness** — Is the inside bright like a sheet?
- **Text / content** — Is there document-like local detail inside?
- **Foreground contrast** — Is the candidate brighter than its surroundings?
- **Boundary evidence** — Do detected edges support its perimeter?
- **Shape geometry** — Do opposite edges and aspect ratio look plausible?

Very large candidates are penalized if they look like the camera frame.

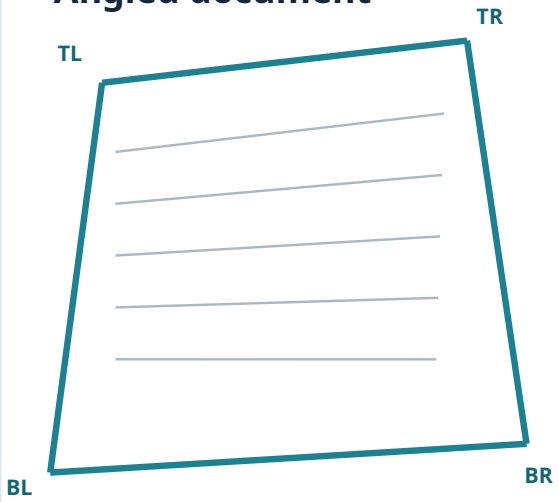
If confidence is low: ask the user to place 4 corners manually. Map detected corners back to the full-resolution photo.

Turn a photographed quadrilateral into a rectangle

The backend uses the four ordered corners to estimate a projective transform and resample the original image.

SOURCE

Angled document



BACKEND TRANSFORM

01 Validate + order corners

02 Estimate output width / height

03 **Compute homography transform**

04 Warp pixels with interpolation

OpenCV: getPerspectiveTransform() + warpPerspective()

RESULT

Rectified scan



Backward mapping samples the source for every output pixel, helping avoid gaps in the corrected image.

Enhance readability and export a useful file

Correction fixes the geometry; enhancement addresses how clearly the content can be read.

Black & White

PHOTOCOPY STYLE



Adaptive local thresholding handles uneven brightness and creates crisp text.

Grayscale

NEUTRAL TONES



Removes color casts while preserving the document's tonal detail.

Contrast Boost

CLAHE



Improves local contrast in shadowed regions without aggressively brightening everything.

EXPORT LAYER

PNG / JPEG

Enhanced image output

→

PDF

ReportLab creates a standard A4 page, scales the image without distortion, and adds margins.

→

DOWNLOAD

All processing remains in the backend API; the client only selects options and displays results.

Designed to recover gracefully when detection is difficult

The project combines validation, geometric checks, transparent processing, and measurable quality indicators.

ROBUSTNESS CHECKS

Corrupt / non-image upload	Reject before processing
Oversized file	Return 413 with guidance
Invalid corner geometry	Reject non-convex, collinear or self-crossing points
Low-contrast scene	Fall back to manual 4-corner selection

WHAT CAN BE EVALUATED

Processing time	How long detection / correction takes
Corner error	Distance from predicted to reference corners
IoU	Overlap between predicted and reference page regions
PSNR / SSIM	Pixel-level and structural similarity indicators

DEMO FLOW

Upload → Detect → Adjust corners → Straighten → Enhance → **Export**

TAKEAWAY **A modular, explainable CV pipeline with a practical human fallback.**